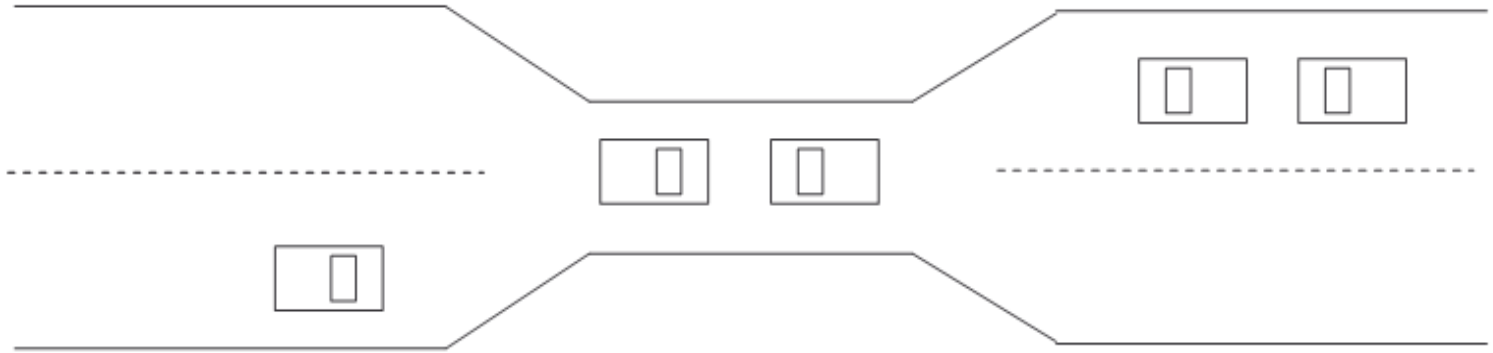
Chapter 5- Deadlock

- Deadlock Overview
- Necessary and Sufficient Deadlock Conditions
- Dealing with Deadlock Problem
  - Deadlock Prevention
  - Deadlock Recovery
  - Deadlock Detection
  - Deadlock Avoidance

Deadlocks

- EXAMPLES IN REAL LIFE:
  - "It takes money to make money".
  - You can't get a job without experience; you can't get experience without a job.
  - The egg and the chick problem.
  - Bridge crossing.



Deadlocks

- **Deadlock** is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.
- In an operating system, a **deadlock** occurs when a **process** or thread enters a waiting state because a requested system resource is held by another waiting **process**, which in turn is waiting for another resource held by another waiting **process**.

Deadlocks

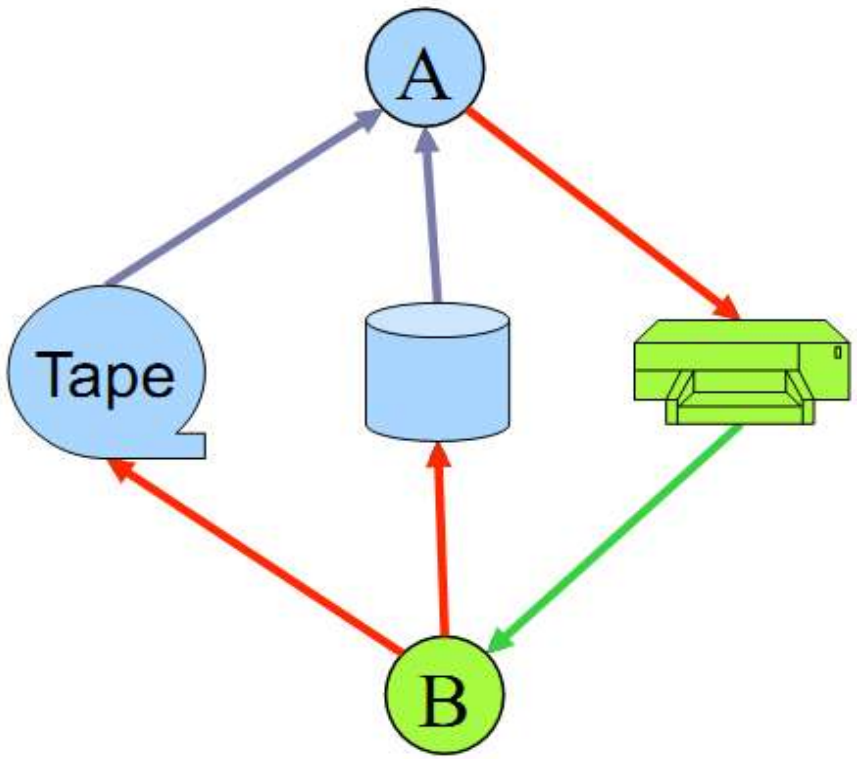
- The cause of deadlocks:
  - Each process needs what another process has. This results from sharing resources such as memory, devices, locks, links.
- Resource to be shared can be
  - Preepmtable --- memory
  - Non-preemptable --- Cd-Drive
- Under normal operation, a resource allocations proceed like this:
  - 1. Request a resource (suspend until available if necessary)
  - 2. Use the resource.
  - 3. Release the resource.

Deadlock- Necessary Conditions

1. Mutual exclusion
   - One or more than one resource must be held by a process in a non-sharable (exclusive) mode.
2. Hold and Wait
   - A process holds a resource while waiting for another resource.
3. No Preemption
   - There is only voluntary release of a resource - nobody else can make a process give up a resource.
4. Circular Wait
   - Process A waits for Process B waits for Process C .... waits for Process A.

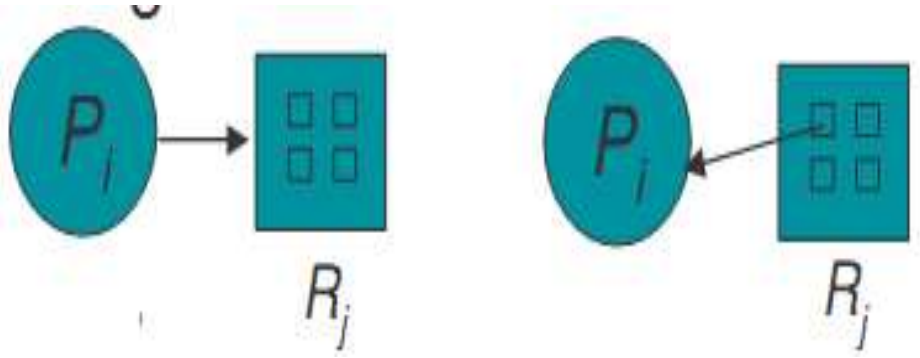
Deadlock- Resource Allocation a visual example

- A utility program
  - Copy a file from tape to disk
  - Print the file to printer
- Resources
  - Tape
  - Disk
  - Printer
- A deadlock
  - A holds tape and disk, then requests for a printer
  - B holds printer, then requests for tape and disk

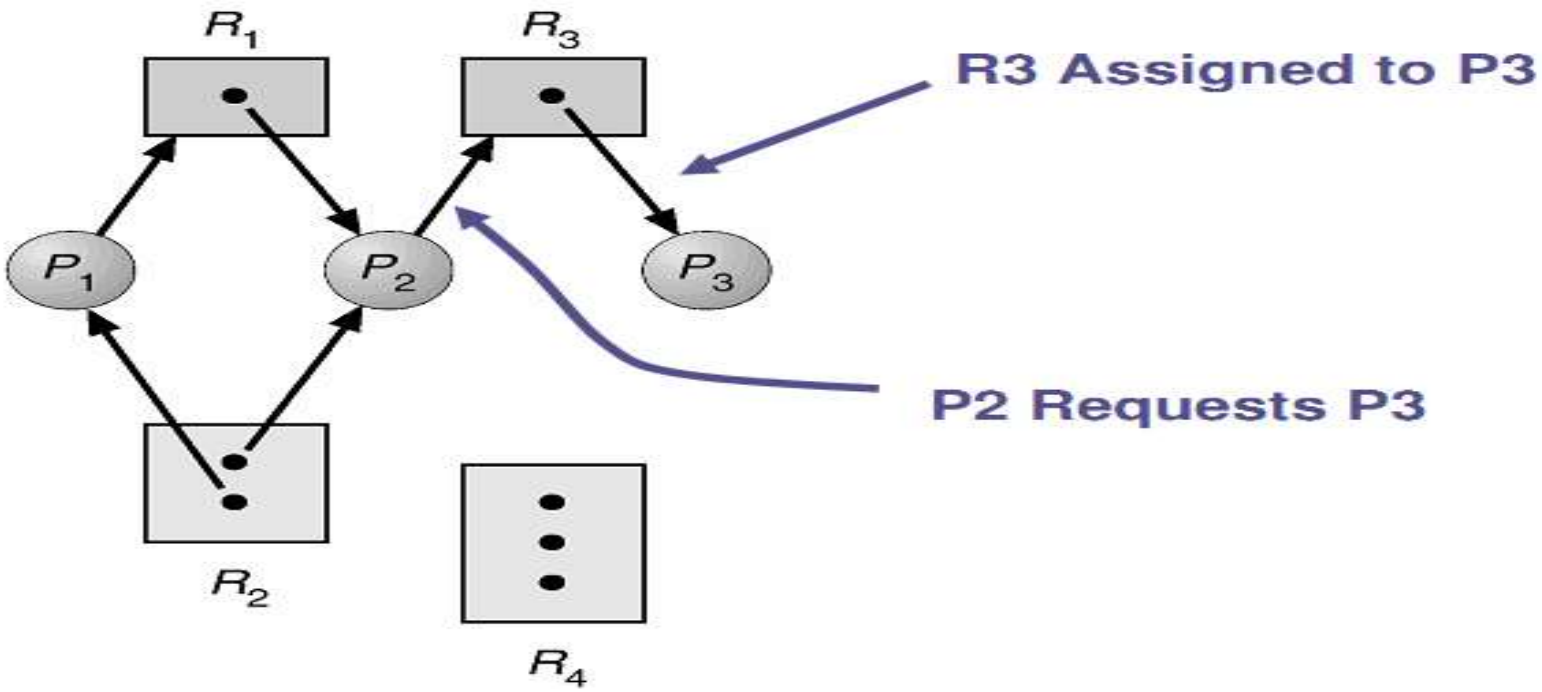


Deadlock- Resource Allocation Graph

- A mathematical way to determine if a deadlock has, or may occur.
  - **G = (V, E)** The graph contains nodes and edges.
  - **V** – Nodes consist of processes **= { P1, P2, P3, ...}** and resource types **{ R1, R2, ...}**
  - **E** – Edges are **(Pi, Rj) or (Ri, Pj)**



Deadlock- Resource Allocation Graph



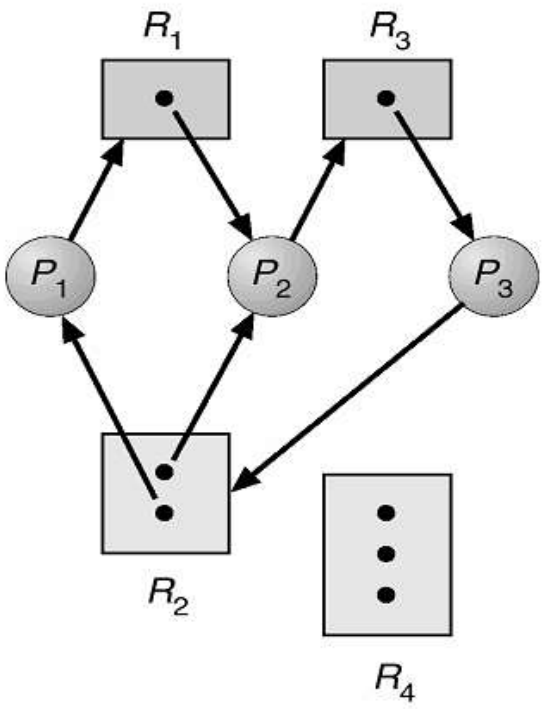
- *Set E contains …*
- *Set V contains….*
- *Set G contains….*
- *How many of instances of each resource are there?*
- *What is each process holding and requesting?*

Deadlock- Resource Allocation Graph

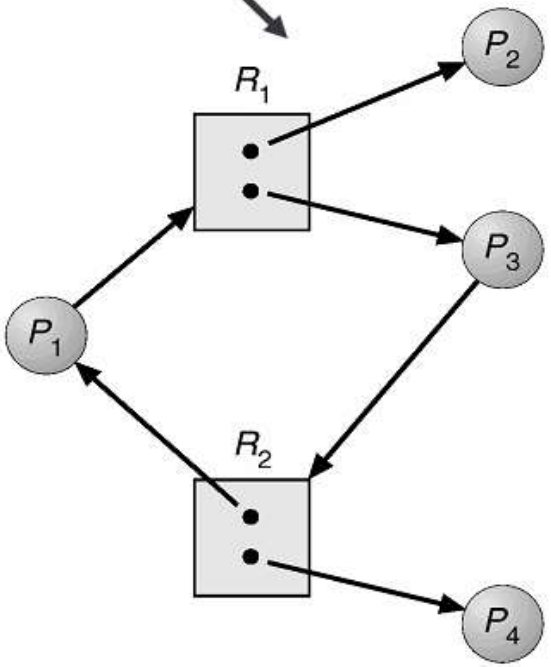
- If the graph contains no cycles, then no process is deadlocked.
- If there is a cycle, then:
  - a) If resource types have multiple instances, then deadlock MAY exist.
  - b) If each resource type has 1 instance, then deadlock has occurred.

Deadlock- Resource Allocation Graph…

Resource allocation graph with a deadlock.



Resource allocation graph with a cycle but no deadlock.



Deadlock- How to handle….General Strategies

- **Ignore the problem**
  - It is user's fault

Most OS use this.

- **Prevention**
  - Negate one of the four conditions
- **Recovery**
  - Fix the problem afterwards

Allow deadlock to happen

- **Detection and Dynamic Avoidance**
  - Careful allocation

Ensure deadlock never occurs

Deadlock… How to handle … Ignore the problem

- **The ostrich algorithm**

  "stick your head in the sand and pretend there is no problem at all"

  - Different people reacts differently.

- What happens when the OS ignores the deadlock???

Deadlock- How to handle….. Ignore the Problem…

- The OS kernel locks up
  - Reboot
- Device driver locks up
  - Remove the device
  - Restart
- An application hangs ("not responding")
  - Kill the application and restart
  - Familiar with this?
- An application ran for a while and then hang
  - Checkpoint the application
  - Change the environment (reboot OS)
  - Restart from the previous checkpoint

Deadlock- How to handle….. Prevention…

- **Do not allow one of the four conditions to occur.**
- **Mutual exclusion:** Ensuring that only one process can access a resource at a time.
  - Allow only if it's absolutely necessary.
  - Shared entities (read only files) don't need mutual exclusion (and aren't susceptible to deadlock.)
- **Hold and wait:** Ensuring that a process does not hold any resources when waiting for another resource.
  - Two ways of doing this—
    - 1. Only request resources when have none and release before asking another.
    - 2. Atomically acquire all resources at once ex. Using a single lock variable
  - Utilization is low, starvation is possible. ***(dark side)***

Deadlock- How to handle….. Prevention…

- **No preemption:**
  - **Allow preemption** - if a needed resource is held by another process, which is also waiting on some resource, steal it. Otherwise wait.
  - Release any resource already being held if the process can't get an additional resource.
  - Utilization is low, starvation is possible and sometimes impossible to pre-emption. ***(dark side)***
- **Circular wait:** Ensuring that there is no circular chain of processes waiting for resources.
  - Global Numbering of resources
  - Only request in ascending order most of the time but descending also prevent deadlock.
  - Only one resource at a time for a process. If ask another– release the first one.

Deadlock- How to handle….. Prevention…

- Each of these prevention techniques may cause a decrease in utilization and/or cause starvation.
  - For this reason, prevention isn't necessarily the best technique.
  - Prevention is generally the easiest to implement.

Decreased Utilization: Certain prevention techniques employ strict rules that force processes to wait even when the shared resource they need is available, simply because the *potential* for a deadlock exists under a different sequence of events.

Starvation: This occurs when a process is repeatedly denied access to a resource needed to proceed, even though other processes are using and releasing it.

Recovery…. Recovery through…

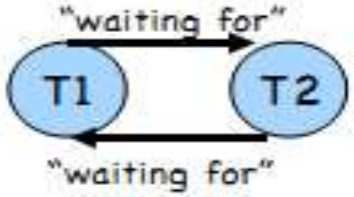
- **Pre-emption**
  - Resources are preempted from the processes involved in the deadlock, preempted resources are allocated to other processes so that there is a possibility of recovering the system from deadlock. In this case, the system goes into starvation.
  - Some times manual intervention is possible
- **Rollback**
  - Processes **check pointed** periodically
  - State of a process can be written to file so that it can be restarted later.
    - Memory image and resource states at different time intervals or points.

Recovery…. Recovery through…

- **Through killing of processes**
  - killing all the process involved in the deadlock. Killing process one by one. After killing each process check for deadlock again keep repeating the process till system recover from deadlock.
  - Rebooting and restart
  - Kill processes in a cycle until the cycle breaks.

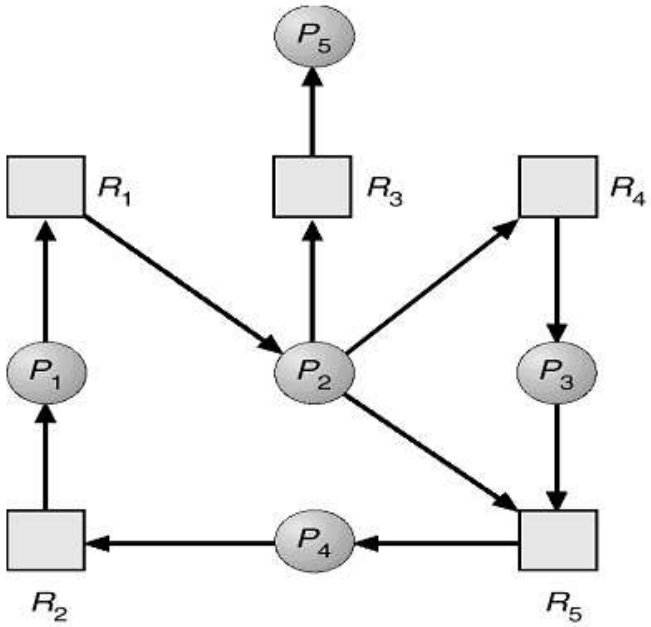
Detection

- **Need an algorithm that determines if deadlock occurred.**
- **Also need a means of recovering from that deadlock.**
- **single instance of a resource type**
  - Wait-for graph == remove the resources from the usual graph and collapse edges.
  - An edge from p(j) to p(i) implies that p(j) is waiting for p(i) to release.

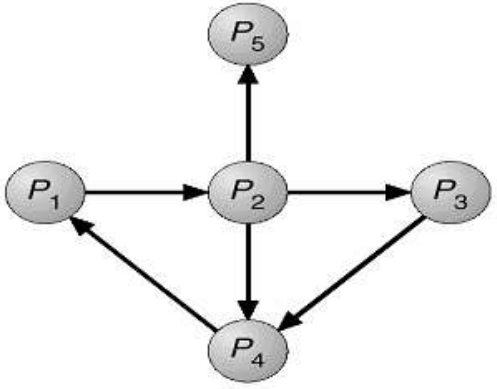


- **Multiple instance of a resource type**
  - Use safe and unsafe state algorithm (Dijkstra's-Banker's algorithm)

Detection..



(a)



(b)

"Wait-for" Graph

Deadlock- How to handle….. Deadlock Avoidance

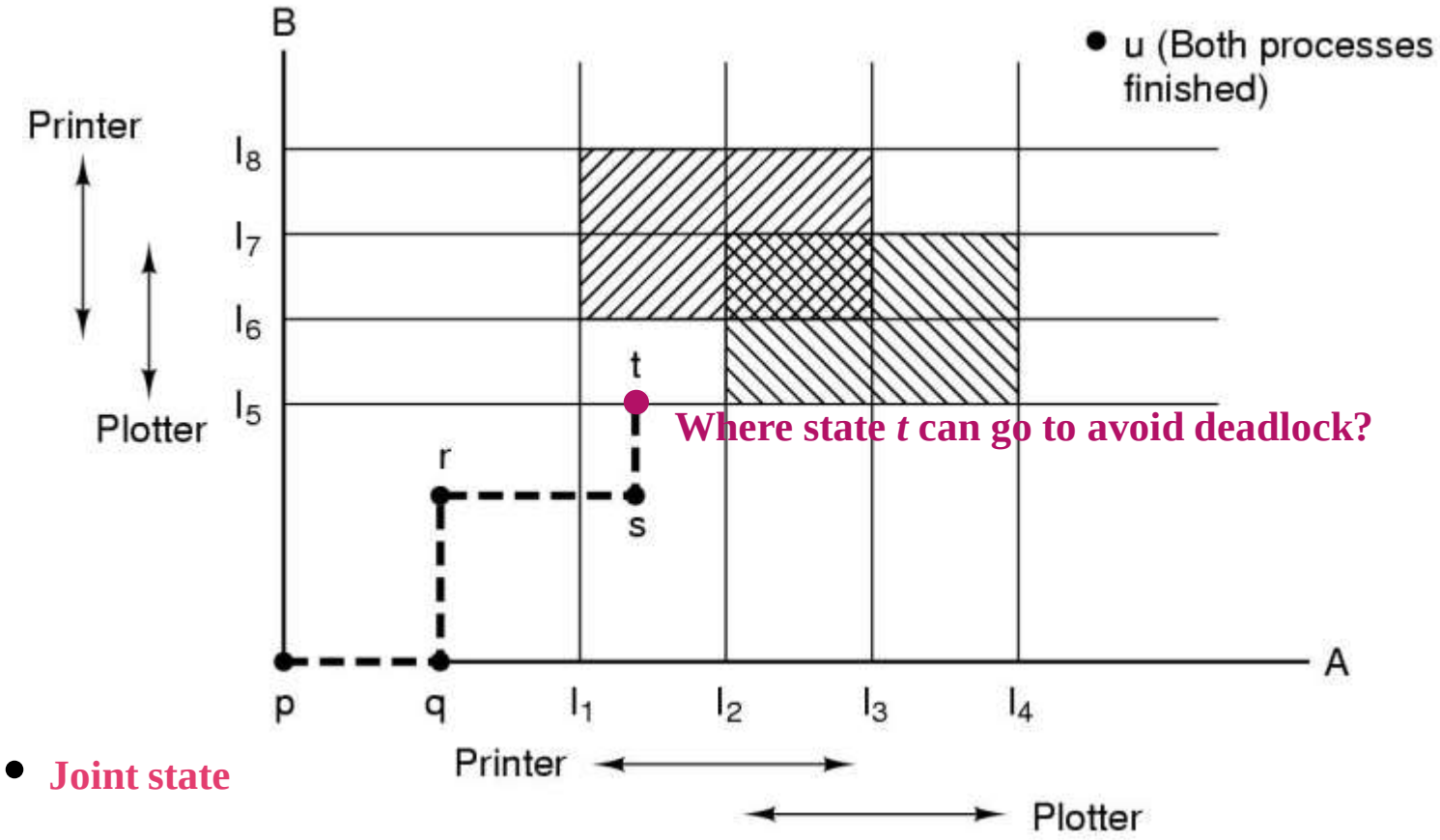
- If we have prior knowledge of how resources will be requested, it's possible to determine if we are entering an "unsafe" state.
- Possible states are:
  - **Deadlock**-No forward progress can be made.
  - **Unsafe state** -A state that may allow deadlock.
  - **Safe state** -A state is safe
    - enough resources for the first to finish
- The rule is simple: If a request allocation would cause an unsafe state, do not honor that request.
- NB: All deadlocks are unsafe, but all unsafe states are NOT deadlocks.

Deadlock- How to handle…..
Deadlock Avoidance

- The Banker's algorithm sometimes referred to as avoidance algorithm or Deadlock algorithm was developed by Edsger Dijkstra (another of Dijkstra's algorithms!).
- It tests the safety of allocation of predetermined maximum possible resources and then makes states to check the deadlock condition.

Deadlock- How to handle….. Deadlock Avoidance.. Resource trajectory

- Trajectories move only right and up!
  - Programs do not run backwards
  - All paths must be horizontal or vertical, never diagonal (if 1 CPU)



- Joint state

Two process resource trajectories

Safe and Unsafe states

- Let's assume a very simple model: each process declares its maximum needs.
- In a case where we have a total of 10 instances.
- In this case, algorithms exist that will ensure that no unsafe state is reached.

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3

(a)

	Has	Max
A	3	9
B	4	4
C	2	7

Free: 1

(b)

	Has	Max
A	3	9
B	0	–
C	2	7

Free: 5

(c)

	Has	Max
A	3	9
B	0	–
C	7	7

Free: 0

(d)

	Has	Max
A	3	9
B	0	–
C	0	–

Free: 7

(e)

Safe and Unsafe states..

- the system is not in a safe state!!!!

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3

(a)

	Has	Max
A	4	9
B	2	4
C	2	7

Free: 2

(b)

	Has	Max
A	4	9
B	4	4
C	2	7

Free: 0

(c)

	Has	Max
A	4	9
B	—	—
C	2	7

Free: 4

(d)

Deadlock- How to handle…. Deadlock Avoidance… Safety Algorithm

Banker's Algorithm

- Multiple instances.
- Each process must a priori claim maximum use.
- When a process requests a resource it may have to wait.
- When a process gets all its resources it must return them in a finite amount of time.

Deadlock- How to handle…. Deadlock Avoidance… Safety Algorithm

- *A method used to determine if a particular state is safe.*
- *It's safe if there exists a sequence of processes such that for all the processes, there's a way to avoid deadlock*

- The algorithm uses these variables:

Need[I] – the remaining resource needs of each process.

Available/Work - Temporary variable – how many of the resource are currently available.

Finish[I] – flag for each process showing we've analyzed that process or not.

need [I]= available + allocated[0] + .. + allocated[I-1] <- Sign of success

Deadlock- How to handle…. Deadlock Avoidance… Safety Algorithm

1. *Initialize Work[]= available[]*

   Initialize finish[i] = false, for i = 1,2,3,..n

2. *Find an i such that:*

   finish[i] == false and need[i] <= available

   If no such i exists, go to step 4.

3. *Work[]= work[]+ allocation[i]*

   finish[i] = true

   goto step 2

4. *if finish[i] == true for all i, then the system is in a safe state.*

Deadlock- How to handle…. Deadlock Avoidance… Safety Algorithm

- Let's do these examples: Consider a system with: five processes, P0-P4,
- Three resource types, A, B, C.
- Type **A has 10 instances,**
  B has 5 instances,
  C has 7 instances.
- At time **T0** the following snapshot of the system is taken.
- Is the system in a safe state?

	Allocation	Max	Available
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Deadlock- How to handle…. Deadlock Avoidance… Safety Algorithm

Do these examples:

Now try it again with only a slight change in the request by P1.

- P1 requests one additional resource of type A, and two more of type C.
- Request1 = (1,0,2).
- Is Request1 < available? And is the system in a safe state.

Exercise

- Can request for (3,3,0) by P4 be granted?
- Can request for (0,2,0) by P0 be granted?

End of Chapter Five